

Prg1

```
from sklearn.datasets import fetch_california_housing
import matplotlib.pyplot as plt, seaborn as sns
```

```
d = fetch_california_housing(as_frame=True).frame
print("Dataset Sample:\n", d.head())
```

```
d.hist(figsize=(15,12), bins=30); plt.suptitle("Histograms"); plt.show()
sns.boxplot(data=d); plt.title("Box Plots"); plt.show()
```

```
print("\nOutlier Analysis using IQR method:")
```

```
for c in d:
```

```
    Q1,Q3 = d[c].quantile([.25,.75]); IQR = Q3-Q1
```

```
    o = d[(d[c]<Q1-1.5*IQR)|(d[c]>Q3+1.5*IQR)]
```

```
    print(f"\n{c}\nMean:{d[c].mean():.2f}, Median:{d[c].median():.2f}, Std dev:{d[c].std():.2f}\nNumber of Outliers:{len(o)}")
```

Prg2

```
import matplotlib.pyplot as plt, seaborn as sns
```

```
from sklearn.datasets import fetch_california_housing
```

```
d = fetch_california_housing(as_frame=True).frame
```

```
c = d.corr()
```

```
print("Correlation matrix:\n", c)
```

```
sns.heatmap(c, annot=True, fmt=".2f", cmap="coolwarm")
```

```
plt.show()
```

```
sns.pairplot(d, corner=True)
```

```
plt.show()
```

Prog 3

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
i=load_iris(); s=StandardScaler().fit_transform(i.data); p=PCA(2); X=p.fit_transform(s)
```

```
[plt.scatter(X[i.target==j,0],X[i.target==j,1]) for j in range(3)]
```

```
plt.show(); print(p.explained_variance_ratio_, sum(p.explained_variance_ratio_))
```

Prog4

```
import pandas as pd
```

```
d = pd.DataFrame({
    'Weather':['Sunny','Sunny','Rainy','Sunny'],
    'Temperature':['Warm','Warm','Cold','Warm'],
    'Humidity':['Normal','High','High','High'],
    'Wind':['Strong','Strong','Strong','Weak'],
    'Play Tennis':['Yes','Yes','No','Yes']
})
```

```
print("Training Data\n", d)
```

```
X, y = d.iloc[:, :-1].values, d.iloc[:, -1].values
```

```
h = X[y=='Yes'][0].copy()
```

```
for i in range(len(X)):
```

```
    if y[i]=='Yes':
```

```
        for j in range(len(h)):
```

```
            if h[j] != X[i][j]:
```

```
                h[j] = '?'
```

```
print("\nFinal Hypothesis\n", h)
```

Prog 5

```
import numpy as np, matplotlib.pyplot as plt
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
np.random.seed(42)
```

```
X = np.random.rand(100,1)
```

```
y = np.where(X<=0.5,'Class1','Class2').ravel()
```

```
f,a = plt.subplots(3,3,figsize=(12,8))
```

```
for i,k in enumerate([1,2,3,4,5,20,30]):
```

```
    m = KNeighborsClassifier(k).fit(X[:50],y[:50])
```

```
    print(f'k={k} Accuracy: {m.score(X[50:],y[50:]):.2f}')
```

```
    a.flat[i].scatter(X[50:],y[50:])
```

```
    a.flat[i].scatter(X[50:],m.predict(X[50:]),marker='x')
```

```
    a.flat[i].set(title=f'k={k}')
```

```
plt.tight_layout(); plt.show()
```

Prog6

```
import numpy as np, matplotlib.pyplot as plt
```

```
x=np.linspace(0,10,100)
```

```
y=np.sin(x)+np.random.normal(0,0.2,100)
```

```
def lwr(x0,t=0.5):
```

```
    w=np.exp(-(x-x0)**2/(2*t*t))
```

```
    X=np.c_[np.ones(len(x)),x]
```

```
    th=np.linalg.pinv(X.T@np.diag(w)@X)@X.T@np.diag(w)@y
```

```
    return [1,x0]@th
```

```
yp=[lwr(i) for i in x]
```

```
plt.scatter(x,y,s=10)
```

```
plt.plot(x,yp,'r')
```

```
plt.show()
```

Prog 7

```
import numpy as np, matplotlib.pyplot as plt  
from sklearn.datasets import fetch_california_housing  
from sklearn.linear_model import LinearRegression  
from sklearn.preprocessing import PolynomialFeatures
```

```
x,y=fetch_california_housing(return_X_y=True)
```

```
m=LinearRegression().fit(x,y)  
p=m.predict(x)
```

```
plt.scatter(y,p,s=5)  
plt.show()
```

```
x=np.arange(1,20).reshape(-1,1)  
y=x.ravel()**2+np.random.randint(10,50,19)
```

```
poly=PolynomialFeatures(2)  
xp=poly.fit_transform(x)
```

```
m.fit(xp,y)  
plt.scatter(x,y)  
plt.plot(x,m.predict(xp),'r')  
plt.show()
```

Prog8

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
```

```
x,y=load_breast_cancer(return_X_y=True)
```

```
x1,x2,y1,y2=train_test_split(x,y,test_size=0.2)
```

```
m=DecisionTreeClassifier().fit(x1,y1)
```

```
print("Accuracy:",m.score(x2,y2))
```

```
plot_tree(m,max_depth=2,filled=True)
```

```
plt.show()
```

Prog 9

```
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
import matplotlib.pyplot as plt
```

```
x,y=fetch_olivetti_faces(return_X_y=True)
```

```
x1,x2,y1,y2=train_test_split(x,y,test_size=0.2)
```

```
m=GaussianNB().fit(x1,y1)
```

```
p=m.predict(x2)
```

```
print("Accuracy:",m.score(x2,y2))
```

```
for i in range(5):
```

```
    plt.subplot(1,5,i+1)
```

```
    plt.imshow(x2[i].reshape(64,64),cmap='gray')
```

```
    plt.title(p[i])
```

```
    plt.axis('off')
```

```
plt.show()
```

Prog 10

```
from sklearn.datasets import load_breast_cancer
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
x,_=load_breast_cancer(return_X_y=True)
```

```
x=StandardScaler().fit_transform(x)
```

```
k=KMeans(n_clusters=2).fit(x)
```

```
plt.scatter(x[:,0],x[:,1],c=k.labels_)
```

```
plt.scatter(k.cluster_centers_[:,0],
            k.cluster_centers_[:,1],
            c='red',marker='x',s=200)
```

```
plt.show()
```